

Savitch's Theorem 강의록

서울대학교 컴퓨터공학부 이태영

7. 공간 복잡도

우리는 지금까지 시간 복잡도에 대해 알아보았다. 시간도 중요한 자원이지만, 우리에게는 시간만 부족한 것이 아니다. 우리는 아쉽지만 한정된 돈으로 인해 제한된 메모리를 갖고 프로그램을 작성해야 하므로 우리가 만든 알고리즘이 얼마나 많은 공간을 차지하는지를 알아보자.

7.1 복잡도 분석

정의 7.1. 항상 정지하는 결정론적 TM M 에 대해, M 의 공간 복잡도(space complexity)는 함수 $f : \mathbb{N} \rightarrow \mathbb{N}$ 로 나타낸다. 이때 $f(n)$ 은 길이가 n 인 입력에 대해 헤드가 테이프에서 접근하는 칸의 최대 개수를 의미한다.

정의 7.2. 모든 상황에 대해 정지하는 비결정론적 TM M 에 대해, M 의 공간 복잡도는 M 의 모든 상황 중 헤드가 접근하는 테이프의 최대 개수를 의미한다.

공간 복잡도 또한 시간 복잡도처럼 점근적 표기로 표현하는 것이 유용하다.

정의 7.3. 함수 $f : \mathbb{N} \rightarrow \mathbb{R}^+$ 에 대해 공간 복잡도 모임(space complexity class)은 다음과 같이 정의된다.

- $\text{SPACE}(f(n))$ 은 공간 복잡도가 $O(f(n))$ 인 결정론적 튜링 기계로 결정할 수 있는 언어들의 모임이다.
- $\text{NSPACE}(f(n))$ 은 공간 복잡도가 $O(f(n))$ 인 비결정론적 튜링 기계로 결정할 수 있는 언어들의 모임이다.

예제 7.4. 앞 장에서 정의한 언어 **SAT**가 $\text{SPACE}(O(n))$ 임을 보여라.

해설. 우리는 **SAT** 정도는 당연히 다항 시간 내에 풀지 못할 것이라고 믿고 있기 때문에 공간 복잡도도 어려울 것이라고 예상하지만, 공간은 시간과 달리 재활용해서 사용할 수 있다. 다음과 같은 튜링 기계 M 을 구상하자.

1. 입력 $\langle \phi \rangle$ 에 대해 ϕ 의 변수 $x_1, \dots, x_k$ 의 진리 부여치를 테이프에 적는다.
2. 1에서 적은 진리 부여치에 따라 ϕ 의 값을 계산하고 만약 1이 나오면 받아들이고, 0이 나오면 다른 진리 부여치 값을 적은 뒤 이를 반복한다.
3. 만약 전부 0이 나오면 받아들이지 않는다.

M 은 당연히 $O(k)$ 의 공간을 사용하고 $k < n$ 이므로 M 은 $O(n)$ 에 속한다는 것을 알 수 있다.

예제 7.5. 다음과 같은 언어 ALL_{NFA} 를 생각하자.

$$ALL_{NFA} = \{ \langle A \rangle \mid A \text{는 NFA이고 } L(A) = \Sigma^* \}$$

$\overline{ALL_{NFA}} \in NSPACE(n)$ 임을 보여라.

시간 복잡도를 다룰 때는 비결정론적 튜링 기계를 결정론적 튜링 기계로 다루는 일은 매우 어려운 일이었다. 하지만 공간 복잡도의 관점에서는 훨씬 적은 공간만으로도 이를 다룰 수 있다.

정리 7.6. (사비치 정리, Savitch's theorem) $f(n) \geq n$ 인 함수 $f : \mathbb{N} \rightarrow \mathbb{R}$ 에 대하여 다음이 성립한다.

$$NSPACE(f(n)) \subseteq SPACE((f(n))^2)$$

증명. A 를 결정하는 비결정론적 튜링 기계 N 에 대해 $N \in NSPACE(f(n))$ 이라고 하자. 이제 A 를 결정하는 결정론적 튜링 기계 M 을 구성하자. M 을 만드는 데 있어서 가장 핵심적인 알고리즘인 CANYIELD를 구성하자. CANYIELD는 N 이 어떤 상황에서 다른 상황까지 정해진 스텝 안에 진행될 수 있는지 판단하는 알고리즘이다. 즉, $CANYIELD(c_1, c_2, t)$ 는 N 의 상황 c_1 에서 t 번의 스텝 안에 c_2 까지 진행되는 것이 가능한지 판단하는 알고리즘이다. 편의성을 위해 우리는 $t = 2^k$ 라고 가정하자. CANYIELD는 다음과 같이 구성될 수 있다.

1. 만약 $t = 1$ 이라면 $c_1 = c_2$ 이거나 N 의 전이 관계 중 $c_1 \vdash_N c_2$ 가 되는 규칙이 있는지 찾고 있으면 받아들이고 없다면 받아들이지 않는다.
2. 만약 $t > 1$ 이라면 N 의 각 상황 c_m 에 대해 공간 $f(n)$ 을 사용하여 3을 계산한다.
3. $CANYIELD(c_1, c_m, \frac{t}{2}), CANYIELD(c_m, c_2, \frac{t}{2})$ 를 계산한 뒤, 둘 다 받아들이면 받아들인다.
4. 만약 3에서 모든 상황에 대해 받아들이지 않으면 받아들이지 않는다.

이제 CANYIELD를 이용해 N 을 흉내내는 M 을 정의하자. 일단 먼저 편의성을 위하여 N 의 계산이 끝나면 테이프의 내용을 지우고 가장 왼쪽으로 헤드를 이동한 뒤 정지하는 상황 c_{accept} 에 들어간다고 하자. 또한, 입력 w 에 대해 N 의 시작 상태를 c_{start} 라고 하자. $n = |w|$ 라고 할 때, N 이 $f(n)$ 만큼의 테이프를 쓸 때 가질 수 있는 상황의 개수가 $2^{df(n)}$ 을 넘지 않도록 상수 d 를 적절히 결정하자.¹ 그럼, N 이 최대 $2^{df(n)}$ 스텝만큼만 움직이므로 M 을 다음과 같이 구상할 수 있다.

1. 입력 w 에 대해 $CANYIELD(c_{\text{start}}, c_{\text{accept}}, 2^{df(n)})$ 을 계산한다.

CANYIELD는 재귀적으로 구성되므로 이를 (c_1, c_2, t) 를 저장하기 위한 스택이 필요하다. N 은 최대 $O(f(n))$ 만큼의 테이프를 사용하므로 (c_1, c_2, t) 의 크기는 $O(f(n))$ 이다. $t = 2^{df(n)}$ 이라 했고, t 는 재귀 한 번마다 절반이 되므로 스택의 최대 길이는 $O(\log 2^{df(n)}) = O(f(n))$ 이다. 따라서 총 필요한 스택의 크기는 $O((f(n))^2)$ 이다.² □

¹ 이는 테이프 알파벳과 상태의 개수에 의존한다.

² 근데 문제가 있다. 여기서 우리는 이미 N 이 $f(n)$ 만큼의 공간을 사용한다는 사실을 알고 있다고 가정하고 시작했다. 만약 이를 모르면 어떻게 할까? 연습문제 7.1로 가자.

7.2 PSPACE

정의 7.7. **PSPACE**는 다음과 같이 정의된다.

$$\text{PSPACE} = \bigcup_k \text{SPACE}(n^k)$$

NPSPACE도 정의하고 싶지만, 어차피 사비치 정리에 의해 $\text{NPSPACE} = \text{PSPACE}$ 이므로 따로 정의하지 않는다.

정리 7.8. $P \subseteq \text{PSPACE}$ 이다.

증명. 시간보다 많이 공간을 사용할 수 없기 때문에 자명하다. □

정리 7.9. $\text{NP} \subseteq \text{PSPACE}$ 이다.

증명. 자명하다. □

현재까지 우리가 알아낸 바를 정리하면 다음과 같다.

$$P \subseteq \text{NP} \subseteq \text{PSPACE} = \text{NPSPACE}$$

미해결 문제 7.10. $\text{NP} = \text{PSPACE}$ 인가?

미해결 문제 7.11. $P = \text{PSPACE}$ 인가?

7.3 PSPACE-완전

우리는 NP-완전을 정의함으로써 아마도 NP와 P가 같지 않을 것이라는 강력한 증거를 주었다. 여기서도 똑같이 PSPACE-완전을 정의하자.

정의 7.12. 다음을 만족할때 언어 B 를 **PSPACE-완전**(PSPACE-complete)이라고 한다.

1. $B \in \text{PSPACE}$
2. 모든 $A \in \text{PSPACE}$ 에 대해 A 는 B 로 다항시간 변환 가능하다.

여기서 B 가 2만 만족하면 B 를 **PSPACE-하드**(PSPACE-hard)라고 한다.

여기서 왜 다항 공간 변환이 아니라 다항 시간 변환을 이용해서 정의하는 것일까? 완전 문제들은 어떤 복잡도 클래스에서 가장 어려운 문제들의 예시이다. 같은 복잡도 클래스 내 다른 문제들이 ‘쉽게’ 그 문제로 변환될 수 있기 때문이다. 따라서 우리가 완전 문제를 ‘쉽게’ 풀 수 있으면 다른 문제들도 ‘쉽게’ 풀 수 있어야 한다. 따라서 변환은 원래 복잡도 클래스 그 자체보다 쉬워야 한다. 앞으로 완전 문제들을 몇 개 더 정의할 것인데, 완전 문제에서의 변환은 항상 그 복잡도 클래스와 비슷하거나 그보다 더 쉬울 것이다.

예제 7.13. 어떤 부울식 ϕ 에 대해 부울식 내 모든 변수가 양화사 $\forall, \exists$ 에 의해 양화되면 이를 **전부 양화된 부울식**(fully quantified Boolean formula)이라고 한다. 예를 들어 다음 부울식은 전부 양화된 부울식이다.

$$\phi = \forall x \exists y [(x \vee y) \wedge (\bar{x} \vee \bar{y})]$$

이때 문제 **TQBF**는 다음과 같이 정의된다.

$$\text{TQBF} = \{\langle \phi \rangle \mid \phi \text{는 전부 양화된 부울식이고 참이다.}\}$$

TQBF는 PSPACE임을 보여라.

해설. 다항 공간만을 사용하는 튜링 기계 M 은 다음과 같다.

1. 입력 $\langle \phi \rangle$ 에 대해 양화사가 없으면 참 거짓을 바로 판단할 수 있다. 참이면 받아들이고 거짓이면 받아들이지 않는다.
2. 만약 ϕ 가 $\exists x, \psi$ 꼴이면 ψ 에 대해 $x = 0$ 일 때와 $x = 1$ 일 때를 재귀적으로 알고리즘을 호출하고 둘 중 하나라도 받아들이면 받아들이고, 둘 다 받아들이지 않으면 받아들이지 않는다.
3. ϕ 가 $\forall x, \psi$ 꼴이면 ψ 에 대해 $x = 0$ 일 때와 $x = 1$ 일 때를 재귀적으로 알고리즘을 호출하고 둘 다 받아들이면 받아들이고, 그 이외의 경우는 받아들이지 않는다.

재귀적으로 호출할 때 최대 재귀 호출 깊이는 전체 변수의 수를 넘지 않는다. 각 호출에서 변수 하나의 값만 저장해두면 되므로 변수 개수 m 에 대해 $O(m)$ 의 공간만을 사용한다.

정리 7.14. TQBF는 PSPACE-완전이다.

7.4 L과 NL

우리는 지금까지 입력 스트링의 길이로 인해 $f(n) \geq n$ 인 경우만 봤다. 그러나 우리가 꼭 이러한 계산 모델만 생각할 이유는 없다. 예를 들어, 컴퓨터가 블루레이 디스크로 입력을 받는다고 하자. 그럼 컴퓨터의 메인 메모리보다 더 큰 크기의 입력이 들어오지만, 실제로 메인 메모리를 다 사용하지는 않을 것이다. 또한, 우리는 그 블루레이 디스크에 새로운 내용을 덧쓸 수 없고 오직 읽을 수밖에 없다. 이러한 경우를 고려하기 위해, 이제 계산 모델을 조금 수정해서 $f(n) < n$ 인 경우도 고려해보도록 하자.

정리 7.15. 읽기 전용 2-테이프 튜링 기계는 다음과 같이 정의된다.

1. 첫 번째 테이프에 입력 스트링이 들어오고 이 테이프는 수정할 수 없다. 입력 전용 테이프(read-only tape)이라고도 한다.
2. 두 번째 테이프는 비어있고, 수정할 수 있다. 작업용 테이프(work tape)이라고도 한다.

정의 7.16. 읽기 전용 2-테이프 튜링 기계의 공간 복잡도(space complexity) $f(n)$ 은 길이가 n 인 입력에 대해 헤드가 두 번째 테이프에서 접근하는 칸 최대 개수를 의미한다.³

예제 7.17. 어떤 1-테이프 튜링 기계 M 과 이와 동등한 읽기 전용 2-테이프 튜링 기계 M' 에 대해 M 이 PSPACE인 것과 M' 이 PSPACE인 것이 동치임을 보여라.

앞으로 이 장에서 사용하는 계산 모델은 읽기 전용 2-테이프 튜링 기계이다. $f(n) \geq n$ 인 경우는 별 다를 바가 없다는 것을 알았으니, $f(n) < n$ 인 경우를 알아보기 위해 다음을 정의하자.

³첫 번째 테이프는 어차피 읽기 전용이므로 고려하지 않는다.

정의 7.18. L 은 다음과 같이 정의된다.

$$L = \text{SPACE}(\log n)$$

정의 7.19. NL 은 다음과 같이 정의된다.

$$NL = \text{NSPACE}(\log n)$$

왜 $\sqrt{n}$, $(\log n)^2$ 같은 함수가 아니라 굳이 $\log n$ 으로 정의했는지 궁금할 수 있는데, 꽤나 많은 컴퓨터 과학적으로 의미있는 문제들이 $O(\log n)$ 공간에 풀리기 때문이다. 그 예들을 알아보자.

예제 7.20. 언어 $A = \{0^n 1^n \mid n \geq 0\}$ 은 L 이다.

해설. 0과 1을 읽으면서 0과 1의 개수를 두 번째 테이프에 이진수로 기록하면 된다. 그럼 $O(\log n)$ 만을 이용해서 결정할 수 있으므로, $A \in L$ 이다.

예제 7.21. 앞 장에서의 PATH 문제에 대해, $\text{PATH} \in NL$ 임을 보여라.

해설. 그래프를 탐색할 때 앞 장의 해설에서 썼던 방식은 공간을 $O(n)$ 만큼 사용하기 때문에 의미가 없다. 따라서 비결정론적인 방식으로 이를 탐색하고자 한다. 탐색을 할 때, 전체 경로가 아닌 현재 탐색하고 있는 정점만을 기록하면 된다. 이제 비결정론적으로 그 정점에서 간선을 따라 다른 정점을 탐색하고, 원래 있던 기록(출발했던 정점)은 지우고 현재 탐색하고 있는 정점으로 덧씌운다. 그래서 이 중에 최종 정점에 도달하게 되면 받아들이고, 정점들의 개수 이상으로 탐색을 많이 하면 받아들이지 않는다.

정의 7.22. M 이 읽기 전용 2-테이프 튜링 기계라고 하자. 입력 w 에 대해 M 의 **상황**(configuration)은 현재 상태, 작업 테이프, 두 테이프에 대한 헤드의 위치로 결정된다. 이때 w 는 상황에 포함되지 않는다.

정리 7.23. M 이 읽기 전용 2-테이프 튜링 기계라고 하자. M 이 길이가 n 인 입력 n 에 대해 $f(n)$ 공간만큼 사용한다고 할 때, w 에 대한 M 의 최대 시간 복잡도는 $n2^{O(f(n))}$ 이다.

증명. 먼저 작업용 테이프에서 나타날 수 있는 테이프의 내용의 가짓수는 어떤 상수 b 에 대해 $b^{f(n)}$ 로 나타낼 수 있다. 또한 헤드의 가짓수는 먼저 입력 전용 테이프에서 n 가지, 쓰기 전용 테이프에서 $f(n)$ 가지가 가능하므로 총 가능한 상황의 수는 $nf(n)b^{f(n)} = n2^{O(f(n))}$ 이다. 따라서 최대 시간 복잡도 또한 $n2^{O(f(n))}$ 이다. $\square$

7.5 NL-완전

일단 $L \subseteq NL$ 임은 자명하다. 그럼 $L \supseteq NL$ 일까? 예제 7.21을 통해 $\text{PATH} \in NL$ 임을 알았지만, 우리는 아직 $\text{PATH} \in L$ 인지 아닌지 모른다. 이 문제 또한 P-NP 문제처럼 컴퓨터 과학에서 아직까지 풀리지 않은 난제들 중 하나이다.

미해결 문제 7.24. $L = NL$ 인가?

따라서 P-NP 문제를 다룰 때와 같이 여기서도 똑같이 NL-완전을 다루고자 한다.

정의 7.25. 로그 공간 변환기(log space transducer)는 읽기 전용 입력 테이프, 쓰기 전용 출력 테이프, 읽기/쓰기 용 작업 테이프가 있는 3-테이프 튜링기계이다. 쓰기 전용 테이프에 있는 헤드는 왼쪽으로 움직일 수 없다. 작업 테이프는 $O(\log n)$ 개의 테이프만 사용할 수 있다. 함수 $f: \Sigma^* \rightarrow \Sigma^*$ 에 대해 로그 공간 변환기 M 이 w 를 입력 테이프를 통해 입력 받았을 때 $f(w)$ 를 출력 테이프에 적고 정지한다면 M 이 f 를 계산한다고 한다. 이러한 M 이 존재하면 f 를 **로그 공간 계산 가능한 함수**(log space computable function)라고 한다. 언어 A 를 B 로 변환가능한 로그 공간 계산 가능한 함수 f 가 존재하면 A 를 B 로 **로그 공간 변환가능**(log space reducible)이라고 하고, $A \leq_L B$ 와 같이 적는다.

정의 7.26. 언어 B 가 다음을 모두 만족하면 **NL-완전**(NL-complete)라고 한다.

1. $B \in \text{NL}$
2. 모든 $A \in \text{NL}$ 에 대해 $A \leq_L B$ 를 만족한다.

2만 만족하면 B 를 **NL-하드**(NL-hard)라고 한다.

정리 7.27. $A \leq_L B$ 이고 $B \in L$ 이면 $A \in L$ 이다.

증명. 일단은 생략한다. 다항 시간 변환에서 사용한 방식을 하면 좋겠으나, 문제는 $f(w)$ 가 로그 공간보다 큰 공간을 차지할 수도 있어서 이를 그대로 사용할 수는 없다. 대신에, □

정리 7.28. PATH는 NL-완전이다.

증명. 예제 7.21에서 PATH는 NL임을 보였으므로, PATH가 NL-하드임을 보이기만 하면 된다.

먼저 $O(\log n)$ 공간에 A 를 계산하는 비결정론적 튜링기계가 있다고 하자. 우리는 A 의 입력 w 를 로그 공간 안에 $\langle G, s, t \rangle$ 로 변환할 것이다. 이때 G 는 방향 그래프로 M 이 w 를 받아들일 때만 s 에서 t 로 가는 경로가 존재하도록 할 것이다.

G 의 각 정점은 M 이 w 를 받아들였을 때의 상태이다. 상태 c_1, c_2 에 대해 c_1 에서 c_2 로 한 번에 전이할 수 있으면 (즉, M 의 전이 관계에 의해 전이 가능한 상태면) G 에 (c_1, c_2) 간선을 만든다. s 는 M 이 w 를 입력받았을 때의 시작 상태이다. M 은 일반성을 잃지 않고 받아들이는 상태가 하나라고 가정하자. 이렇게 G 를 만들면 PATH를 통해 s 에서 t 로 가는 경로가 존재하는지 검사해 M 이 w 에 대해 받아들이지 않는지 알 수 있다.

그럼 이제 이러한 변환이 로그 공간에 가능하다는 것을 보이자. w 에 대해 $\langle G, s, t \rangle$ 을 출력하는 로그 공간 변환기 T 를 만들자. G 는 노드들을 나열하고 이후에 (c_1, c_2) 꼴로 이루어진 간선들을 나열한다. 먼저 G 의 노드들의 경우, 각 노드의 크기는 어떤 상수 c 에 대해 최대 $c \log n$ 이다. G 의 노드로 가능한 $c \log n$ 크기 이하의 문자열을 찾기 위해서 T 는 $c \log n$ 크기 이하의 문자열을 하나하나 만들고 이것이 M 이 w 를 입력받고 나서의 적절한 상황인지를 검사한다. 제대로 된 상황이면 이를 출력 테이프에 적는다. 이는 로그 공간 안에 해결할 수 있다.

간선의 경우도 유사하게 $2c \log n$ 크기 이하의 문자열을 하나하나 만들고 이것이 M 에 대해 제대로 된 전이인지를 검사한다. 제대로 된 전이면 이를 출력 테이프에 적는다. 이 또한 로그 공간 안에 해결할 수 있다. □

정리 7.29. $\text{NL} \subseteq \text{P}$

증명. 먼저 공간 $f(n)$ 을 사용하는 튜링 기계의 경우, 정리 7.23에 의해 최대 $n2^{O(f(n))}$ 의 시간을 사용한다. 따라서 로그 공간을 사용하는 변환기의 경우 최대 다항 시간 안에 정지한다. 따라서 NL인 문제는 모두 PATH로 다항 시간 안에 변환 가능하고, PATH는 다항 시간 안에 해결 가능하므로 $NL \subseteq P$ 를 증명할 수 있다. $\square$

미해결 문제 7.30. $P = NL$ 인가?

미해결 문제 7.31. $P = L$ 인가?

7.6 연습문제

연습문제 7.1. 정리 7.6의 증명을 완성하라. (힌트: 튜링 기계가 시간을 완전히 포기하고 $f(n) = 1, 2, 3, \dots$ 를 하나하나 차근차근 해보면 된다.)